

HaluGuard Project Summary

What We Are Doing

HaluGuard is a research project about making repository-level code generation more reliable by reducing **code hallucinations**.

In practical terms, the problem is this:

- Large language models can often write code that looks convincing.
- But when they work inside a real repository, they frequently invent things that are not actually present in the codebase.
- These mistakes include nonexistent imports, wrong function names, wrong method calls, wrong argument usage, and code that runs but produces the wrong behavior.

HaluGuard tries to solve that problem by changing how context is selected and how generation is corrected.

Instead of asking, "Which code chunks look most similar to the query?", HaluGuard asks, "Which code chunks are most likely to prevent a hallucination?"

That is the core idea of the project.

The Main Research Claim

The project is built around the hypothesis that:

Context chosen for its hallucination-prevention value is more useful than context chosen only for semantic similarity.

The system therefore combines three main ideas:

1. A learned scorer that ranks repository chunks by hallucination-prevention potential.
2. A type-aware router that boosts the kinds of context most likely to prevent a specific failure.
3. An execution feedback loop that runs generated code, reads the actual error, and retries with better-targeted context.

What We Did

This repository implements the full workflow needed to explore that idea:

- Built the HaluGuard package for repository-level context ranking and inference.
- Added utilities to generate supervision from RepoBench using `gold_snippet_index`.
- Added frozen-encoder embedding helpers for CodeBERT and notebook workflows that also support UniXcoder variants.
- Implemented a legacy HCCS scorer plus a broader model zoo with seven ranking architectures.
- Implemented training helpers for example-level ranking, hard negatives, curriculum learning, focal-style losses, and checkpointing.
- Implemented rule-based type routing before generation and after execution failures.

- Implemented an execution feedback loop that retries generation after sandboxed execution errors.
- Implemented retrieval baselines including BM25, cosine similarity, lexical overlap, random ranking, no-context, full-context, and oracle gold-only selection.
- Implemented evaluation helpers for generation metrics and retrieval metrics.
- Created notebook workflows for data preparation, training, evaluation, and pipeline demonstration.
- Added tests for the critical CPU-safe parts of the system.

What Problem We Are Solving

The project focuses on **repository-level code generation**, where a model must generate code that works with files elsewhere in a repository.

That setting is harder than standalone code generation because the model often needs:

- the real function definition from another file,
- the correct import path,
- the correct class or method name,
- the correct signature or argument structure,
- or the correct logic implied by tests and surrounding code.

If the model does not see the right supporting context, it tends to guess. Those guesses are what HaluGuard treats as hallucinations.

Hallucination Types Used in the Project

The project standardizes four hallucination categories in `haluguard/hccs.py` through the shared `HallucinationType` enum.

1. Resource hallucination

This is when the model imports a module, package, or resource that does not exist.

Typical Python errors:

- `ImportError`
- `ModuleNotFoundError`

2. Naming hallucination

This is when the model uses the wrong variable, function, class, or attribute name.

Typical Python errors:

- `NameError`
- `AttributeError`
- `UnboundLocalError`

3. Mapping hallucination

This is when the model uses a real API incorrectly, such as the wrong argument types, wrong keys, or wrong indexing.

Typical Python errors:

- `TypeError`
- `KeyError`
- `IndexError`

4. Logic hallucination

This is when the code executes but the behavior is still wrong.

Typical Python errors:

- `AssertionError`
- `ValueError`
- `RuntimeError`
- related runtime failures such as `ZeroDivisionError`, `RecursionError`, or `StopIteration`

Benchmarks and Data

The project is centered around two benchmark datasets.

RepoBench v1.1

Primary benchmark:

- Dataset: `tianyang/repobench_python_v1.1`
- Split: `cross_file_first`
- Size: 8,033 examples

Why it matters:

- Each example gives the current partially written file as `cropped_code`.
- It also provides candidate cross-file context chunks.
- Most importantly, it includes `gold_snippet_index`, which tells us which chunk was actually needed for the correct next line.

That label is what makes the HaluGuard training setup possible without manually annotating examples.

CrossCodeEval

Secondary benchmark:

- Dataset: `ArtifactAI/cceval`
- Python split

Why it matters:

- It serves as a transfer benchmark.

- It tests whether the retrieval and ranking behavior learned on RepoBench generalizes to another cross-file completion setting.

End-to-End Workflow

The full HaluGuard workflow has two broad phases:

1. Offline preparation and training
2. Inference and evaluation

Phase 1: Offline preparation and training

This phase builds the supervision signals and trains the context scorer.

The steps are:

1. Load benchmark examples.
2. Embed the query code and context chunks with a frozen encoder.
3. Use `gold_snippet_index` to identify the positive chunk for each example.
4. Treat all other chunks as negatives.
5. Build ranking examples or contrastive triplets.
6. Train a scorer to rank the gold chunk above the negatives.
7. Save checkpoints and cached embedding artifacts.

Phase 2: Inference and evaluation

This phase uses the trained scorer to help a code LLM generate the correct next line.

The steps are:

1. Embed the current code and candidate context chunks.
2. Score and rank the chunks.
3. Apply type-router boosts based on likely error modes.
4. Select top-k context chunks.
5. Build a prompt for the generator model.
6. Generate the next line of code.
7. Optionally execute the result in the EFL loop.
8. If execution fails, classify the error and retry with re-ranked context.
9. Measure generation quality and retrieval quality.

Data Preparation in This Repo

The data preparation logic lives in `haluguard/data_pipeline.py`, with the full workflow in `notebooks/01_data_pipeline.ipynb`.

Core data structure

The repo defines a `ContrastiveTriplet` dataclass with:

- `query`
- `positive_context`
- `negative_context`
- `positive_path`
- `negative_path`
- `task_id`
- `gold_snippet_index`

How triplets are created

For each `RepoBench` example:

- `cropped_code` becomes the query.
- `context[gold_snippet_index]` becomes the positive context.
- every other chunk becomes a negative context.

If an example has invalid labels or too few chunks, it is skipped.

What gets saved

The pipeline supports saving and loading triplets in JSONL format.

Expected artifact:

- `data/triplets.jsonl`

The repo also already contains cached embedding artifacts under `data/embeddings/`.

Embedding Strategy

The embedding helpers live in `haluguard/hccs.py`.

Encoder behavior

The project uses frozen code encoders. The encoder is not fine-tuned during scorer training.

Supported usage patterns in the repo:

- CodeBERT-based embeddings in the core package
- notebook workflows for CodeBERT and UniXcoder variants

Query vs context truncation

The code uses different truncation behavior for queries and context chunks.

- Queries default to **left truncation** so the most recent code near the prediction point is preserved.
- Context chunks default to **right truncation** so the beginning of a definition is preserved.

Pooling

The helpers support:

- cls pooling
- mean pooling

Limits

The project convention is:

- CodeBERT input truncation at **512 tokens**
- generation prompt truncation at **2048 tokens**

The Original HCCS Scorer

The legacy scorer in `haluguard/hccs.py` is the original Hallucination Context Contrast Scorer.

Input representation

It builds pair features from query and chunk embeddings:

- query
- chunk
- $|query - chunk|$
- $query * chunk$

This gives the scorer direct access to both the embeddings and their interaction patterns.

Architecture

The legacy HCCS MLP is:

- `Linear(3072, 512)`
- `ReLU`
- `Dropout(0.1)`
- `Linear(512, 1)`
- `Sigmoid`

Training signal

It is trained with contrastive supervision using positive and negative context examples.

The Model Zoo

The more general scorer architectures live in `haluguard/models.py`.

The repo currently supports seven scorer architectures:

1. DualEncoder

- separate query and chunk projections
- cosine similarity scoring
- learned temperature

2. DualEncoderDeep

- deeper query and chunk projections
- cosine similarity scoring
- more capacity than the simple dual encoder

3. ListwiseMLP

- concatenates query and chunk embeddings
- produces per-chunk logits directly
- trained with listwise cross-entropy

4. PairwiseMLP

- pairwise scoring MLP
- used with InfoNCE-style contrastive behavior

5. InteractionMLP

- uses interaction features such as query, chunk, absolute difference, and elementwise product
- higher-capacity scorer for richer ranking behavior

6. BilinearScorer

- low-capacity factorized bilinear scoring
- smallest parameter count among the main architectures

7. EnsembleScorer

- learns a mixture over strong base models
- acts as a learned combination layer over top-performing scorers

The package also exposes `MODEL_REGISTRY`, `build_model`, and parameter-count helpers.

Training Pipeline

The training helpers live in `haluguard/training.py`, and the primary notebook is `notebooks/02_train_hccs.ipynb`.

What the training code does

The training module moves the ranking logic out of notebooks into reusable Python helpers.

It includes:

- example-level ranking data structures
- data splitting helpers
- hard-negative ordering
- training subset selection
- dataloaders for listwise ranking
- unified scorer dispatch
- curriculum learning support
- in-batch negative handling
- focal-style loss helpers
- stochastic weight averaging helpers

Ranking example structure

The key training data structure is `RankingExample`, which stores:

- example id
- query embedding
- all candidate chunk embeddings
- gold chunk index
- hard negative indices sorted by similarity

Negative sampling

The training utilities support:

- hard negatives
- random negatives
- keeping the gold chunk first in sampled candidate sets

Data splits

The repo includes example-level splitting helpers to avoid leakage between train and validation samples.

The default dataset-level split helper creates:

- 70% train
- 15% validation
- 15% test

Notebook-level training behavior

The notebook test coverage confirms that the training notebook includes:

- all six base architectures
- the `EnsembleScorer`

- curriculum training
- early stopping
- checkpoint saving

Project working conventions from the repo context also emphasize:

- batch size **256** for MLP training
- InfoNCE temperature **0.07**
- testing with small subsets before scaling up

Type Router

The rule-based type router lives in `haluguard/type_router.py`.

It is intentionally not learned. The repo treats this mapping as a logical layer rather than something that needs a separate model.

Pre-emptive routing

Before generation, the router looks at `cropped_code` and predicts what kinds of context are likely to help.

Examples:

- method calls like `obj.method()` increase the boost for naming-related context
- import statements increase the boost for resource-related context
- type annotations increase the boost for mapping-related context
- assertions or test-like code increase the boost for logic-related context
- active function or class definitions slightly increase naming-related boosts

Snippet classification

Candidate chunks are classified using lightweight heuristics:

- import-heavy snippets or `__init__` files tend to count as resource context
- function or class definitions tend to count as naming context
- typed signatures are one of the heuristic signals considered for mapping-related context
- test files tend to count as logic context

Post-failure routing

After an execution failure, the router maps the actual Python exception to a hallucination category and applies stronger boosts for that category.

Known error type:

- strong boost for the matched category

Unknown error type:

- mild boost across all categories

Code Generation

Generation helpers live in `haluguard/generate.py`.

Prompt construction

The prompt builder prepends selected cross-file snippets before the current file code.

Typical prompt structure:

- one or more `# Cross-file context:` sections
- optional import statements from the current file
- the current `cropped_code` at the end

The code is placed at the end so generation is conditioned directly on the latest local context.

Generation entry points

The repo provides:

- `generate_next_line` for single-example generation
- `generate_next_line_batch` for batched evaluation

Generator assumptions

The project is designed around Hugging Face causal language models.

The code and project notes expect models such as:

- DeepSeek-Coder
- Qwen2.5-Coder
- similar causal code LMs

Project conventions currently use:

- generation temperature **0.2**
- max selected chunks **5**
- max EFL retries **3**

Execution Feedback Loop

The execution feedback loop is implemented in `haluguard/efl.py`.

This is one of the most important parts of the project.

What it does

The EFL:

1. builds a prompt from the selected context
2. generates code

3. creates a small executable test program
4. runs it in a subprocess with a timeout
5. checks whether the attempt passed
6. if it failed, parses the error and re-ranks context
7. retries until success or the maximum number of iterations is reached

Execution behavior

The repo uses:

- a temporary Python file
- `subprocess.run`
- timeout control
- cleanup of the temp file in a `finally` block

This design is simple enough to work in Colab without Docker, while still giving basic isolation.

Returned structures

The module defines:

- `ExecutionResult`
- `EFLResult`

These store:

- whether execution passed
- stdout and stderr
- parsed error type
- parsed error message
- hallucination classification
- iteration count
- history across attempts

Error parsing

The EFL extracts:

- the exception class name from stderr
- the meaningful error message
- the hallucination category

It then uses router boosts to change the context ranking for the next attempt.

Inference Pipeline

The end-to-end pipeline lives in `haluguard/pipeline.py`.

What the pipeline wraps

It combines:

- a scorer
- a tokenizer
- a frozen encoder
- top-k context selection
- the type router
- and the EFL

What it supports

The pipeline can work with:

- the legacy `HCCSScorer`
- any of the scorer architectures from `haluguard/models.py`

Main pipeline flow

For a single example, the pipeline:

1. embeds the query
2. embeds the candidate snippets
3. scores all chunks
4. applies type-router boosts
5. selects top-k chunks
6. passes the selected context into the generation function
7. optionally runs the EFL loop
8. returns the final prediction and related metadata

Baselines

The retrieval and ablation baselines live in `haluguard/baselines.py`.

These are important because the project is not only trying to build a working system. It is also trying to show that HaluGuard is better than simpler retrieval strategies.

Baselines implemented

- **No context**: select nothing
- **BM25**: keyword overlap baseline
- **Jaccard**: token-set overlap baseline
- **Edit similarity**: token-sequence similarity baseline

- **Cosine retrieval**: embedding-space similarity baseline
- **Random ranking**: deterministic random ordering
- **Full context**: provide every chunk
- **Gold only**: oracle baseline that always picks the gold chunk

Why these matter

They provide comparisons against:

- classic lexical retrieval
- standard embedding retrieval
- trivial lower and upper bounds
- and an oracle reference point

Evaluation

Evaluation helpers live in `haluguard/evaluate.py` and `haluguard/retrieval_benchmark.py`. The main notebook is `notebooks/03_evaluation.ipynb`.

Generation metrics

The repo computes:

- **Exact Match (EM)**: exact next-line match
- **Edit Similarity (ES)**: character-level similarity
- **CodeBLEU**: code-aware similarity, with graceful fallback behavior when the optional dependency is missing

Retrieval metrics

The repo computes:

- **Accuracy@k**
- **MRR**

Query views

The retrieval benchmark code supports:

- `full`
- `last3`

That means the system can evaluate retrieval using either the entire cropped code or only the last three lines as the query view.

Candidate buckets

The repo groups examples into:

- easy for 5-9 candidate chunks

- hard for 10 or more candidate chunks

This makes it easier to evaluate whether methods still work when retrieval becomes more difficult.

Artifact naming helpers

The retrieval benchmark module also standardizes:

- query embedding artifact paths
- chunk embedding artifact paths
- metadata artifact paths
- checkpoint naming conventions

Notebook Workflow

The repo is notebook-first. The notebooks are not secondary extras; they are the main research interface.

`01_data_pipeline.ipynb`

This notebook is responsible for:

- loading benchmark datasets
- computing query and chunk embeddings
- supporting both CodeBERT and UniXcoder variants
- saving embedding metadata
- generating training data
- writing cached artifacts for later notebooks

Notebook tests confirm that it includes:

- `codebert with full`
- `codebert with last3`
- `unixcoder with full`
- `unixcoder with last3`

`02_train_hccs.ipynb`

This notebook is responsible for:

- loading precomputed embeddings
- training the model zoo
- saving checkpoints
- early stopping
- curriculum-based ranking training
- ensemble training

`03_evaluation.ipynb`

This notebook is responsible for:

- retrieval evaluation
- generation evaluation
- building paper-style summary outputs
- saving retrieval summaries and result artifacts

`04_pipeline_demo.ipynb`

This notebook is responsible for:

- showing the end-to-end pipeline in action
- running the EFL loop
- comparing outcomes such as when HCCS wins versus cosine retrieval
- basic error analysis

Saved Artifacts and Outputs

The repo expects generated artifacts under `data/`, which is gitignored.

Current artifact types used by the project

- `data/triplets.jsonl`
- `data/embeddings/query_embeddings__*.pt`
- `data/embeddings/chunk_embeddings__*.pt`
- `data/embeddings/*.meta.json`
- `data/embeddings/gold_indices.pt`
- `data/embeddings/test_indices.pt`
- evaluation result files in `data/results/`

Checkpoints

Model checkpoints are expected under:

- `checkpoints/`

The notebook utilities also support saving checkpoints to Google Drive for Colab sessions.

Notebook Utilities

Shared notebook helpers live in `notebooks/utils.py`.

They support:

- GPU detection
- environment summaries

- Google Drive mounting
- Drive path creation
- checkpoint save and load helpers
- JSONL append and counting helpers
- benchmark dataset loading

These helpers are especially important because the project is designed for Colab use, where sessions can disconnect and storage needs to be handled carefully.

Testing and Validation Coverage

The repo includes CPU-friendly tests for important non-GPU logic.

`tests/test_data_pipeline.py`

Validates:

- triplet creation
- handling of invalid gold indices
- negative sampling caps
- JSONL save/load roundtrips
- summary statistics

`tests/test_generate.py`

Validates:

- prompt construction
- multiple snippet insertion
- omission of empty import statements
- placement of `cropped_code` at the end of the prompt

`tests/test_efl.py`

Validates:

- successful execution
- error classification
- import, naming, mapping, and logic failures
- EFL retries
- history tracking
- cleanup of temporary files

`tests/test_hccs_training.py`

Validates:

- embedding truncation behavior
- normalization behavior
- pooling behavior
- pair feature construction
- ranking example building
- data splitting
- hard-negative selection

`tests/test_evaluate.py`

Validates:

- empty metric handling
- CodeBLEU fallback behavior when the dependency is missing

`tests/test_retrieval_benchmark.py`

Validates:

- query view construction
- easy/hard bucket assignment
- ranking order
- monotonicity of accuracy metrics
- ranking table summarization

`tests/test_notebook_retrieval_workflow.py`

Validates:

- notebook code cell syntax
- expected notebook sections and training flow
- presence of architecture coverage
- retrieval sections
- EFL usage in the demo notebook

Current State of the Repo

As it stands now, the repo contains:

- the package implementation
- the model zoo
- the notebook workflow
- training and evaluation utilities
- cached embedding artifacts

- a generated triplet file
- test coverage for the main non-GPU logic

This means the repo is not just an idea sketch. It is already a working implementation scaffold for:

- preparing data
- training scorers
- evaluating retrieval
- evaluating generation
- and demonstrating the full pipeline

What Has Been Implemented vs What Is Expected at Run Time

Already implemented in code

- data structures
- embedding helpers
- ranking models
- training helpers
- routing logic
- generation helpers
- execution feedback loop
- baselines
- evaluation helpers
- notebook utilities
- tests

Expected from the runtime environment

- access to Hugging Face models and datasets
- a GPU-backed environment for practical training speed
- optional CodeBLEU dependency for full metric support
- checkpoint and results generation by running the notebooks

Project Conventions and Important Defaults

The project instructions and codebase conventions emphasize the following defaults:

- Python 3.9+
- use of `typing.Optional`, `typing.List`, and `typing.Dict`
- use of `pathlib.Path`

- dataclasses for key containers
- notebook-first workflow
- test with 10 examples before scaling
- batch size **256**
- InfoNCE temperature **0.07**
- generation temperature **0.2**
- top-k selected chunks **5**
- max EFL retries **3**
- CodeBERT max length **512**
- generation prompt max length **2048**

In One Sentence

HaluGuard is a complete research pipeline for learning which repository context chunks best prevent code hallucinations, using supervised ranking from RepoBench, rule-based routing for hallucination types, and execution-grounded retries to improve next-line code generation.

Short Executive Summary

If someone asks what this project did, the shortest accurate answer is:

HaluGuard builds and evaluates a system that improves repository-level code generation by selecting context based on hallucination-prevention value rather than similarity alone, training ranking models on RepoBench supervision, adding type-aware routing and execution-based retries, and comparing the result against strong retrieval baselines with both retrieval and generation metrics.